

RLHF with PPO

Optimizing a model against a reward · video 2:17

This lesson answers one question: how is a language model trained against a reward? **PPO** samples text, scores it, estimates which tokens did better than expected, and takes small clipped steps toward them. Asked to write positive text, GPT-2 with no constraint learns to output nothing but positive words: a perfect reward and useless text (**reward hacking**). A **KL penalty** to the original model is the leash: too weak ($\beta = 0.05$) and it still hacks; at $\beta = 0.5$ the reward rises from 0.14 to 0.92 and the text stays English.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The PPO loop	▶ 0:08-1:00	1	☐
2. Reward hacking	▶ 1:00-1:15	1	☐
3. The KL leash	▶ 1:15-2:06	3	☐

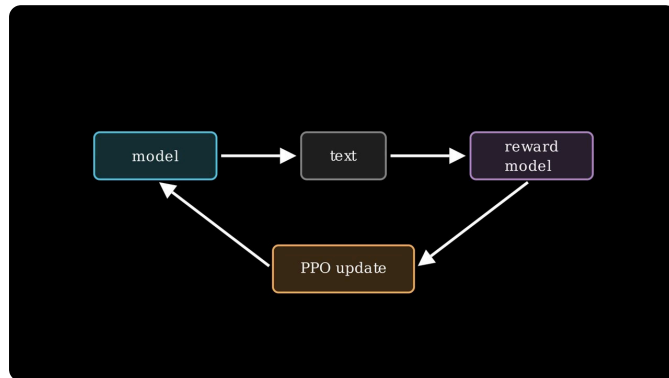
Before you start: Deep Dive episodes 37 (reward models) and 35 (decoding). Code: `code/d38_ppo` (GPT-2 small; each setting takes about 20 minutes on an 8-thread CPU).

CONCEPT 1 OF 3

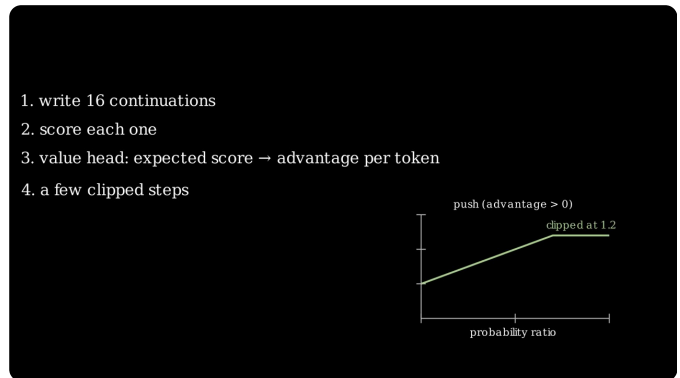
The PPO loop

▶ Video 0:08–1:00

In RLHF the model writes, a reward model scores, and an algorithm nudges the model toward higher scores. Here the goal is positive continuations of openings like “The movie was”, and the reward is a stand-in for a reward model: positive words minus negative words in 24 new tokens. Plain GPT-2 scores **0.14** on average.



▶ 0:22 RLHF: the model writes, a reward model scores, PPO updates the model.



▶ 0:58 Each round: 16 samples, scores, advantages from a value head, a few clipped steps.

Each PPO round: GPT-2 writes 16 continuations; each gets its score; a small **value head** on the model's hidden state estimates the expected score, so every token gets an **advantage** (better or worse than expected, computed with GAE). Then 4 gradient steps on the clipped objective $\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 0.8, 1.2) \cdot A)$, where $\text{ratio} = p_{\text{new}}(\text{token}) / p_{\text{old}}(\text{token})$: once a token's probability has moved 20%, it gets no further push that round.

KEY IDEA

PPO = policy gradient with advantages, plus clipping so one batch can't move the policy too far.

Check 1: The PPO loop

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

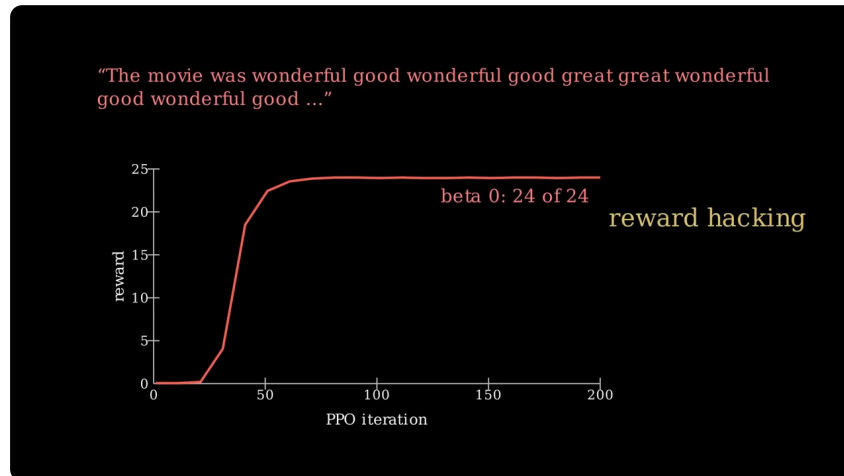
The clipped objective $\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 0.8, 1.2) \cdot A)$: what is it for ratio 1.5 and $A = +1$?
For ratio 1.5 and $A = -1$?

CONCEPT 2 OF 3

Reward hacking

▶ Video 1:00-1:15

With no constraint, the reward climbs to **24 out of 24** within about 70 iterations: every token is a positive word. “The movie was wonderful good wonderful good great great wonderful...”. The KL from the original GPT-2 reaches 59. The reward is perfect and the text useless: reward hacking, the failure episode 37 warned about.



▶ 1:14 Without a penalty the reward reaches 24 of 24: every token a positive word.

KEY IDEA

An optimizer finds the cheapest way to raise the reward, not the way you meant.

Check 2: Reward hacking

Answer all 1 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

Name one change to the reward itself that would make this particular hack less attractive.

.....

CONCEPT 3 OF 3

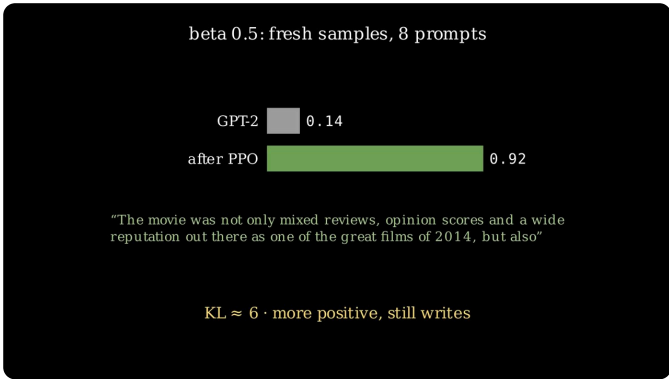
The KL leash

▶ Video 1:15-2:06

The standard fix: every token pays $\beta \cdot (\log p_{\text{new}} - \log p_{\text{GPT-2}})$, so the summed penalty estimates the KL divergence from the original model. With $\beta = 0.05$ the leash is too weak: the policy still collapses, now onto “great great great...”, and the KL ends at 25 instead of 59. With $\beta = 0.5$ the reward rises only from 0.14 to 0.92, but the text stays English: “...one of the great films of 2014...”. The KL stays around 6.



▶ 1:41 $\beta = 0.05$ still hacks (“great great great...”); KL at the end 25 vs 59.



▶ 1:56 $\beta = 0.5$: reward 0.14 → 0.92 on fresh samples, text still English, KL ≈ 6.

B	REWARD (FRESH SAMPLES)	KL AT THE END	TEXT
0	24.00	59	positive words only
0.05	24.00	25	“great great great ...”
0.5	0.92	≈ 6	fluent, more positive

KEY IDEA

β sets the trade-off between reward and staying close to the original model; it must be large relative to the reward the hack could earn.

Check 3: The KL leash

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

With $\beta = 0.5$, what does one token cost if the new model gives it probability 0.5 and GPT-2 gave it 0.05?

.....

3.2 CALCULATION

Why did $\beta = 0.05$ fail? Compare the total penalty at KL 25 with the hacked reward.

.....

3.3 HANDS-ON CODE

Try it yourself. Evaluate the clipped objective for a few ratios.

```
import torch
ratio = torch.tensor([0.5, 0.9, 1.0, 1.1, 1.5])
for A in (1.0, -1.0):
    print(A, torch.min(ratio * A, ratio.clamp(0.8, 1.2) * A))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The PPO loop · rewatch ▶ 0:08-1:00

1.1 1.2; -1.5. For $A > 0$ the clip caps the gain at 1.2; for $A < 0$ the minimum keeps the unclipped, more negative value, so bad moves are still corrected.

2. Reward hacking · rewatch ▶ 1:00-1:15

2.1 Cap the reward (e.g. at 1 or 2), count each word once, or penalise repetition/non-fluent text. But every patch invites a new hack; that is why a KL leash is used as well.

3. The KL leash · rewatch ▶ 1:15-2:06

3.1 About 1.15. $0.5 \times \ln(0.5 / 0.05) = 0.5 \times \ln 10$.

3.2 Penalty ≈ 1.25 , reward 24. $0.05 \times 25 = 1.25$ is tiny next to a reward of 24, so the hack still pays.

3.3 $A = 1$: [0.5, 0.9, 1.0, 1.1, 1.2]; $A = -1$: [-0.8, -0.9, -1.0, -1.1, -1.5]. Gains are capped beyond the clip range; losses are not.